

Sandbox Online Assessment System — Principal Architect Technical Review

Reviewed as: Principal Software Architect / TPM / BA / Engineering Manager

Document reviewed: "Sandbox Online Assessment System — Detailed Technical Specification & Team Work Distribution Matrix" (3 roles, 3 microservices, 6-member work matrix, 14 functional modules)

Basis: Findings are based only on the attached document. Anything marked *Recommendation* is my addition, not something the document states.

1. Requirement Completeness Review

The document is a **module/feature checklist with a team allocation matrix** — it is not an SRS. It lists *what* to build but almost never *how it should behave*.

Critical gaps:

Category	What's Missing	Why It Matters
Business rules	No rules for subscription limits (max HRs, max candidates, max assessments per plan), no negative-marking formula, no passing-score formula, no grace period for exam re-entry	Without numeric rules, backend logic is undefined — every developer will guess differently
Edge cases	No handling for: candidate loses internet mid-exam, browser crash/refresh during exam, duplicate submission, exam time expires while grading in progress, candidate assigned same assessment twice, coding submission with infinite loop	These are the most common real-world failure points in exam platforms
Validations	No field-level validation rules (email format, password policy, file size/type limits for CSV import, min/max marks, min/max test cases)	Needed for both frontend forms and backend DTOs

Category	What's Missing	Why It Matters
Workflows	No candidate re-attempt policy, no HR review/override of AI score, no organization onboarding approval flow, no proration/refund flow on subscription downgrade	These are standard SaaS flows the doc silently assumes away
Error handling	Zero mention of API error contracts, retry policy for Judge0/LLM failures, or what happens if AI proctoring service is down during a live exam	An exam platform cannot fail silently during a live, timed exam
Security	No mention of RBAC matrix (who can see whose data), no data residency/PII handling for candidate video/photos	Proctoring footage is sensitive biometric/PII data — this is a compliance risk
Audit	Zero audit trail requirement beyond violation logs (no admin action log, no "who changed this assessment" trail)	Needed for enterprise HR trust and dispute resolution
Reporting	Report <i>names</i> are listed, but no filters, export formats (beyond "PDF optional"), or scheduling	Reports need date ranges, org-level vs candidate-level scoping
Notifications	Only email is covered; no in-app notifications, no SMS, no notification preferences/opt-out	
Role permissions	Roles are described narratively, no permission matrix (module × action × role)	Needed to actually build RBAC
API contracts	Endpoints are given as wildcard prefixes (<code>/auth/**</code>) with no request/response schemas	Cannot start frontend-backend contract work from this alone

Category	What's Missing	Why It Matters
UI requirements	No wireframes, no field-level form specs, no state definitions (loading/empty/error)	
Database requirements	Table names only, no columns, types, or relationships	
Infrastructure	No mention of hosting, containers, environments (dev/stage/prod), CI/CD	
Deployment	Not mentioned at all	Directly affects whether "12 days" is feasible

Verdict: this reads as a **scope outline for team allocation**, not a build-ready SRS.

2. End-to-End Flow Review

For each core workflow, here's what's documented vs. missing:

Organization Onboarding → HR Provisioning

- Documented: Super Admin creates org, provisions HR.
- Missing: approval step, org verification (domain/email), HR invite-acceptance flow, what happens to existing candidates if org is deactivated.

Candidate Registration/Invitation

- Documented: HR adds candidate; candidate registers/updates profile.
- Missing: whether candidates are pre-created by HR (invite-only) or self-register and get matched to an org; duplicate email handling across multiple orgs; invite expiry.

Question Bank → Assessment Build

- Documented: CRUD for MCQ/coding questions, assessment builder with timer/passing/negative marking.
- Missing: question versioning (editing a question already used in a live assessment), tagging/reuse across assessments, bulk import for question banks (CSV mentioned only for candidates).

Assignment → Exam Lifecycle

- Documented: assign, bulk-assign, notify, reassign, remove.
- Missing: what "remove assignment" does if candidate already started; concurrency handling for auto-save; explicit state machine (Not Started → In Progress → Submitted → Graded → Reviewed).

Exam Execution (MCQ + Coding)

- Documented well at feature level (palette, timer, auto-save, auto-submit).
- Missing: sync protocol between client and server (polling vs. websocket) for auto-save and timer authority (client clock vs. server clock — critical to prevent cheating via clock manipulation).

AI Code Evaluation

- Documented: Judge0 execution, LLM code review, complexity/quality scoring.
- Missing: scoring weight between Judge0 pass/fail and LLM subjective score; timeout/sandbox limits per submission; what happens if Judge0 or the LLM API times out during grading.

AI Proctoring

- Best-documented module in the doc, including violation types and snapshot/video logic.
- Missing: retention policy for photos/videos, consent capture from candidate, escalation flow (does HR get alerted live or only post-exam), false-positive handling.

Result → Report Generation

- Documented: score calculation, pass/fail, PDF (optional).
- Missing: re-grading trigger if HR manually overrides a coding score, report generation timing (real-time vs. batch job).

Payments/Subscription

- Documented: Razorpay integration, plan view/purchase, expiry tracking.
- Missing: webhook idempotency, failed-payment retry, downgrade/proration logic, invoice generation.

Recommendation: every workflow above needs a state-machine diagram and a sequence diagram before backend work starts — none exist in the source document.

3. Database Review

The document names tables per microservice but gives **no columns, types, keys, or relationships**. Rough estimate below:

Table	Purpose	Present in Doc	Missing Detail
users	Auth identity	✓ named	Columns, role FK, org FK
roles	RBAC roles	✓ named	Permission mapping table absent
organizations	Tenant	✓ named	Status, plan FK, settings
subscriptions	Plan lifecycle	✓ named	Plan definition table missing (subscriptions ≠ plans)
payments	Transaction log	✓ named	Currency, gateway refs, refund fields
candidates	Candidate profile	✓ named	Org FK, source/import batch ref
questions	MCQ question	✓ named	Category/tag table missing
mcq_options	MCQ answer options	✓ named	—
coding_questions	Coding problem	✓ named	Test case tables not separated

Table	Purpose	Present in Doc	Missing Detail
test_cases (sample/hidden)	Per coding question	✗ missing	Currently implied inside coding_questions only
assessments	Assessment config	✓ named	—
assessment_questions	Mapping	✓ named	—
assignments	Candidate ↔ assessment	✓ named	—
submissions	MCQ answers	✓ named	—
coding_submissions	Code answers	✓ named	—
results	Final scores	✓ named	—
violations	Proctoring events	✓ named	—
proctoring_logs	Raw events	✓ named	—
ai_evaluations	LLM output	✓ named	—
plans	Subscription tier definitions	✗ missing	Referenced narratively, no table
notifications	Email/notification log	✗ missing	No audit of what was sent/when
audit_logs	Admin/system actions	✗ missing	No traceability for changes

Table	Purpose	Present in Doc	Missing Detail
question_categories/tags	Categorization	✗ missing	"Categorize Questions" feature has no backing table
refresh_tokens	JWT refresh	✗ missing	Refresh mentioned nowhere despite JWT lifecycle claim
org_settings / feature_flags	Tenant config	✗ missing	—
file_uploads / media	CSV, PDFs, snapshots, videos	✗ missing	No central asset table; snapshot/video "paths" stored ad hoc

Estimate: ~19 tables named vs. **~28–32 tables realistically required** for a system with tenancy, audit, notifications, and file management. Junction/lookup/audit tables are almost entirely absent from the document.

4. API Review

APIs are given only as wildcard prefixes, not endpoint-level. Estimating actual REST surface by module (CRUD + search/filter/export where applicable):

Module	Estimated Endpoints Needed	Documented
Authentication	8–10 (register, login, refresh, verify, forgot/reset, change-password, logout, profile)	Prefix only
Organization	6–8 (CRUD, activate/deactivate, list, HR provisioning)	Prefix only

Module	Estimated Endpoints Needed	Documented
Subscription	6–8 (plans list, purchase, verify, renew, cancel, history)	Prefix only
Payment	5–6 (initiate, webhook, verify, history, refund)	Prefix only
Candidate	8–10 (CRUD, search/filter, bulk import, export)	Prefix only
Question Bank	10–12 (MCQ CRUD, coding CRUD, test-case CRUD, tag/search/filter)	Prefix only
Assessment	8–10 (CRUD, publish/unpublish, add/remove questions, clone)	Prefix only
Assignment	6–8 (assign, bulk-assign, reassign, remove, list-by-candidate/assessment)	Prefix only
Exam runtime	8–10 (start, save-answer, mark-for-review, submit, auto-submit, resume, status)	Prefix only
Results	4–6 (get result, HR view, override/re-grade, list)	Prefix only
Reports	6–8 (assessment report, candidate report, stats, PDF export, proctoring report)	Prefix only
AI Evaluation	3–4 (evaluate, get-status, re-evaluate)	Prefix only
Proctoring	5 (snapshot, video-upload, violation-log, get-violations, summary)	Prefix only (named)
Notifications	2–3 (list, mark-read – if in-app is added)	Not covered
Admin/Audit	3–4	Not covered

Estimate: ~90–110 endpoints required for full scope. The document documents 0 at the endpoint-contract level – only 12 route prefixes across 3 services. This is the single largest gap for a development team trying to start work in parallel (6 members working simultaneously with no shared contract will cause severe integration drift).

5. UI/Frontend Review

The document lists screen *names* embedded in the work matrix (e.g., "Login & Register Pages," "Assessment Builder Workflow UI") but provides no detail.

Missing:

- Missing pages: system settings/admin config, notification center, audit log viewer, plan comparison page, invoice/billing history page, 404/permission-denied pages
- Missing dialogs: confirm-delete, session-timeout warning, exam-submit confirmation, camera/mic permission denied fallback
- Missing forms: no field lists for any form (e.g., what exactly is in "Add Candidate" — name, email, phone, dept?)
- Missing validation rules: none specified anywhere in UI context
- Missing navigation: no sitemap or role-based menu structure
- Missing responsive requirements: not mentioned — critical since candidates may take exams on varied devices
- Missing accessibility requirements: not mentioned at all (a11y is often contractually required for enterprise SaaS)
- Missing loading/empty/error states: not specified for any screen

Recommendation: a full screen inventory + low-fidelity wireframes are a prerequisite before frontend estimation is trustworthy.

6. Backend Review

Documented: JWT lifecycle, Spring Security, BCrypt, SMTP, Razorpay SDK, Judge0, LLM integration, auto-save/auto-submit engine, PDF generation.

Missing:

- Transaction handling (e.g., is assignment + email send atomic? what if email fails after DB commit?)
- Exception handling strategy / global error format
- Background job framework (auto-submit at timer expiry, subscription-expiry reminders, video upload processing) — implied but no scheduler/queue technology named
- Queue requirements — proctoring snapshot analysis and video uploads should likely be async but no queue (Kafka/RabbitMQ/SQS) is named

- File storage — where do snapshots/videos/PDFs/CSVs live? No S3/Blob storage mentioned
- Caching — none mentioned; question banks and subscription plans are strong caching candidates
- Logging strategy — beyond "proctoring_logs" table, no application/system logging (ELK, CloudWatch, etc.)
- Retry mechanisms — none for Judge0, LLM, Razorpay webhook, or email delivery failures

7. Security Review

Item	Status in Doc
Authentication	✔ JWT + Spring Security named
Authorization / RBAC	⚠ Roles named, but no permission matrix
Refresh Tokens	✗ Not mentioned despite "JWT lifecycle" claim
Rate limiting	✗ Missing — critical for /auth/login and /judge/run
CSRF	✗ Not mentioned
CORS	✗ Not mentioned
SQL Injection protection	✗ Not mentioned (assumed via ORM, not stated)
XSS protection	✗ Not mentioned (important given rich text in question banks)
Encryption at rest	✗ Not mentioned — relevant for candidate PII and proctoring media
Secrets management	✗ Not mentioned (Razorpay keys, LLM API keys, JWT secret)
Audit logging	⚠ Only proctoring violations logged, not admin/system actions
API security (input validation, payload size)	✗ Not mentioned

Item	Status in Doc
limits)	
File upload validation	✗ Not mentioned — high risk given video/snapshot/CSV uploads
Consent for biometric monitoring	✗ Not mentioned — legal/compliance risk, camera + face detection is biometric data in many jurisdictions

This is the weakest section of the source document relative to what a production system handling biometric proctoring data and payments would require.

8. Integration Review

Integration	Documented Depth	Missing
Razorpay	Payment + webhook mentioned	Webhook signature verification, refund flow, sandbox/live key management
Judge0	Code execution mentioned	Language/runtime versions, sandbox limits, self-hosted vs. cloud instance, timeout handling
AI/LLM (OpenAI/Gemini)	Code review + proctoring vision mentioned	Which model, cost/rate-limit handling, prompt versioning, fallback if API is down
Email (SMTP)	Named per-member	Provider (SES/SendGrid?), template management, bounce handling
Camera/Browser APIs	Feature list given (tab switch, blur, fullscreen)	Browser compatibility matrix, permission-denial UX
PDF generation	"Optional"	Library/service not named

Integration	Documented Depth	Missing
Storage service	Implied by "store snapshot/video path"	No actual service (S3/GCS/Azure Blob) named

Four integrations (Judge0, LLM vision, Razorpay, storage) are the highest-effort, highest-risk items in the whole build, and all four have incomplete specs.

9. Engineering Estimation

Based on the true scope (not the stated 6-person/12-day framing):

Track	Realistic Effort (person-weeks)
Backend (3 microservices)	10–14
Frontend (candidate + HR + admin portals)	8–12
DevOps (multi-service deploy, CI/CD, storage, monitoring)	2–3
QA (functional + security + load, exam platforms need heavy edge-case testing)	4–6
AI/Proctoring tuning (Judge0 sandboxing, LLM prompt tuning, vision accuracy)	3–5
Total	~27–40 person-weeks

Suggested team for a realistic timeline (6–8 weeks): 2–3 backend engineers, 2 frontend engineers, 1 DevOps (part-time), 1–2 QA, 1 engineer focused solely on AI/proctoring integration — i.e., roughly the 6 people named, but over **weeks, not 12 days**.

10. Feasibility Analysis — Can this be built in 12 calendar days?

No, not as full production scope. Reasoning:

- The doc itself implies **19+ DB tables, 3 microservices, 14 functional modules, 3 external integrations (Judge0, LLM, Razorpay), and biometric AI proctoring** — this is enterprise SaaS scope, not an MVP.
- 6 members × 12 days ≈ 72 person-days (~14.4 person-weeks) of raw capacity — **roughly half** of the ~27–40 person-weeks estimated above, even before QA/bug-fix/deployment buffers.
- AI proctoring (vision-based face/phone detection) and AI code evaluation (LLM-based) both require tuning cycles that don't compress well under time pressure — false positives/negatives need real testing data.

Scenario	Outcome
Best Case (skilled team, prior boilerplate/starter kit reused, all integrations pre-vetted, no scope changes)	A thin vertical slice is achievable: auth + org + one full exam flow (MCQ only, no coding/proctoring) working end-to-end. Full scope: not achievable.
Realistic Case	Core auth, org, candidate, question bank, and basic MCQ assessment flow functional; coding + AI evaluation + AI proctoring left partially stubbed or mocked; payments likely sandbox-only.
Worst Case	Integration friction (Judge0 sandboxing, LLM latency/cost, Razorpay webhook issues) consumes most of the timeline; only auth + CRUD screens are demo-ready; exam runtime and AI features not functional.

Recommendation: if 12 days is a hard constraint, explicitly descope to an MVP (e.g., MCQ-only exams, no AI proctoring, mocked payments) and treat AI code evaluation + proctoring as Phase 2.

11. Risk Analysis

Risk Level	Modules	Why
High	AI Proctoring (camera/vision), AI Code Evaluation (LLM +	External API dependency + no documented fallback/retry;

Risk Level	Modules	Why
	Judge0), Payments (Razorpay webhooks)	proctoring also carries legal/compliance exposure (biometric data, consent)
Medium	Exam Runtime (timer sync, auto-save, auto-submit), Question Bank/Assessment Builder, Reports (PDF generation)	Concurrency and state-machine complexity; PDF generation "optional" but likely expected by HR clients
Low	Authentication, Organization Management, Candidate CRUD	Well-understood, standard CRUD + JWT patterns, low novelty

12. Review Quality Verdict

⚠ **Needs Minor Changes** — *(caveat: this rating reflects the document as a scope/allocation matrix, not a formal SRS; if it's expected to function as a build-ready SRS, the correct verdict is ❌ Not Ready)*

Reasoning:

- As a **work-distribution and scope-alignment document for a 6-person team**, it's usable — modules, ownership, and rough table/API groupings are clear enough to start sprint planning.
- As an **SRS to hand to developers for direct implementation**, it is not ready: no field-level specs, no API contracts, no ERD, no business rules, no non-functional requirements (security, performance, compliance), and no wireframes.
- Before development starts, the minimum bar should be: (1) ERD with real columns/keys, (2) API contract doc (even just request/response shapes per endpoint), (3) a permission matrix, (4) explicit business rules for scoring/timers/subscriptions, (5) a security requirements checklist.

13. Final Executive Summary

Metric	Estimate
Modules (documented)	14
Database tables (documented / realistically required)	~19 / ~28–32
APIs (documented as contracts / realistically required)	0 contract-level / ~90–110 endpoints
UI screens (named / realistically required incl. states & admin)	~20 named / ~35–45
External integrations	4 (Razorpay, Judge0, LLM/AI vision, Email) — plus implied storage service (5th, undocumented)
Missing requirements identified	40+ across completeness, security, DB, API, and UI sections above
Key technical debt risks	No API contracts → integration drift across 6 parallel workstreams; no ERD → schema churn mid-build; no security spec → biometric/PII compliance exposure; no retry/fallback design → live-exam outages on 3rd-party API failure
Timeline estimate (full scope)	6–8 weeks with the given 6-person team, vs. the requested 12 days
12-day feasibility	Not feasible for full scope. Feasible only for a narrowly descoped MVP (see Section 10)
Overall confidence score	35/100 for "ready to build full scope as-is in 12 days"; 70/100 for "usable as a starting point for a properly scoped SRS + sprint plan"

Note: This review is based solely on the two documents provided (the review-prompt PDF and the pasted specification text). No assumptions were made

about undocumented client requirements — every "missing" item above is an absence in the source text, not a guess about what the client actually wants.